Notification Service

Detail Architecture & Component Interaction

Document version: 1.0 · Multi-channel notification (SMS, Email, Push) for internal and mobile users.

Table of contents

1. Overview & scope
2. High-level architecture
3. Component detail
4. Data stores & models
5. Interaction: Send immediate notification
6. Interaction: New mobile subscriber → email to internal
7. Interaction: Scheduled notifications (Scheduler Service)
8. Notification Service layers
9. API summary
10. Deployment & scalability

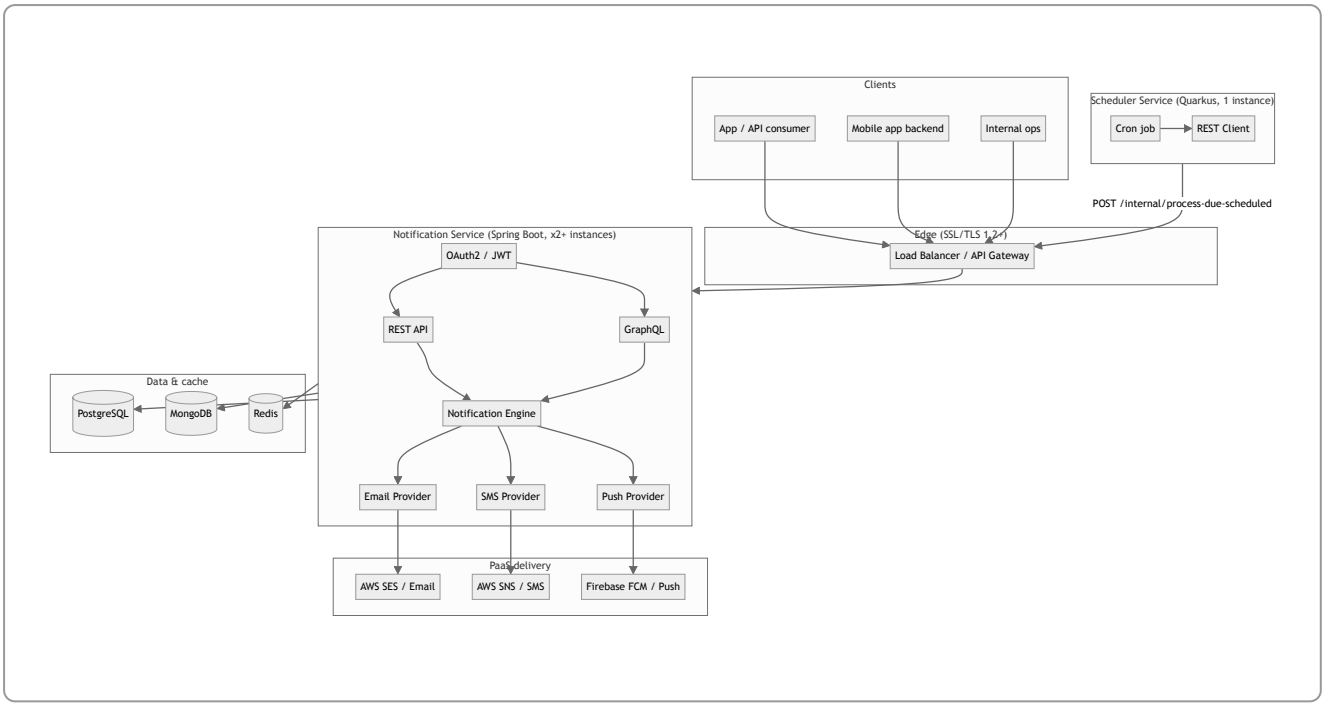
1. Overview & scope

The Notification Service delivers messages through **SMS**, **Email**, and **Push Notifications**, targeting **internal users** (operators, customer services) and **mobile application users**. It is designed for high availability (minimum two active instances) and uses PaaS for delivery (e.g. AWS SES/SNS, Firebase FCM).

Key capabilities:

- Delivery channels: SMS, Email, Push
- Audiences: INTERNAL, MOBILE_APP
- Immediate and scheduled notifications
- REST and GraphQL APIs; OAuth2 + JWT
- Separate Scheduler Service (Quarkus) for cron-driven processing

2. High-level architecture



Traffic reaches the Notification Service via a load balancer (HTTPS). The Scheduler Service calls the internal endpoint to process due scheduled notifications. The Notification Service uses PostgreSQL for core data, MongoDB for event logs, and Redis for idempotency and rate limiting. Delivery is delegated to PaaS (SES, SNS, FCM).

3. Component detail

3.1 Notification Service (Spring Boot)

Component	Responsibility
REST API	Send notification, create/list/cancel scheduled, subscriptions, templates, internal process-due-scheduled
GraphQL	Mutations: sendNotification, createScheduledNotification, cancelScheduledNotification; queries: scheduledNotifications, subscriptions
OAuth2 / JWT	Validate tokens for REST/GraphQL; internal endpoint optionally protected by API key
Notification Engine	Resolve recipients, render templates, call channel providers, write audit events, idempotency
SMS / Email / Push providers	Abstractions over AWS SNS/SES, Firebase FCM (or stubs in dev)

3.2 Scheduler Service (Quarkus)

Single responsibility: run a cron (default every minute), call `POST /api/v1/internal/process-due-scheduled` on the Notification Service with optional `X-Internal-Api-Key`. No direct database access.

3.3 Data stores

Store	Purpose
PostgreSQL	Templates, subscriptions, internal users, scheduled notifications
MongoDB	Append-only notification events (audit, analytics)
Redis	Idempotency keys (TTL), rate-limit counters

4. Data stores & models

4.1 PostgreSQL (core domain)

Entity	Key fields
notification_template	id, channel (SMS/EMAIL/PUSH), name, body_template, subject_template, external_template_id
notification_subscription	id, user_id, channel, audience_type (INTERNAL/MOBILE_APP), destination (phone/email/token)
internal_user	id, email, roles (e.g. operators, support)
scheduled_notification	id, template_id, channel, audience_type, audience_filter, scheduled_at, timezone, status (PENDING/SENT/CANCELLED), context_json

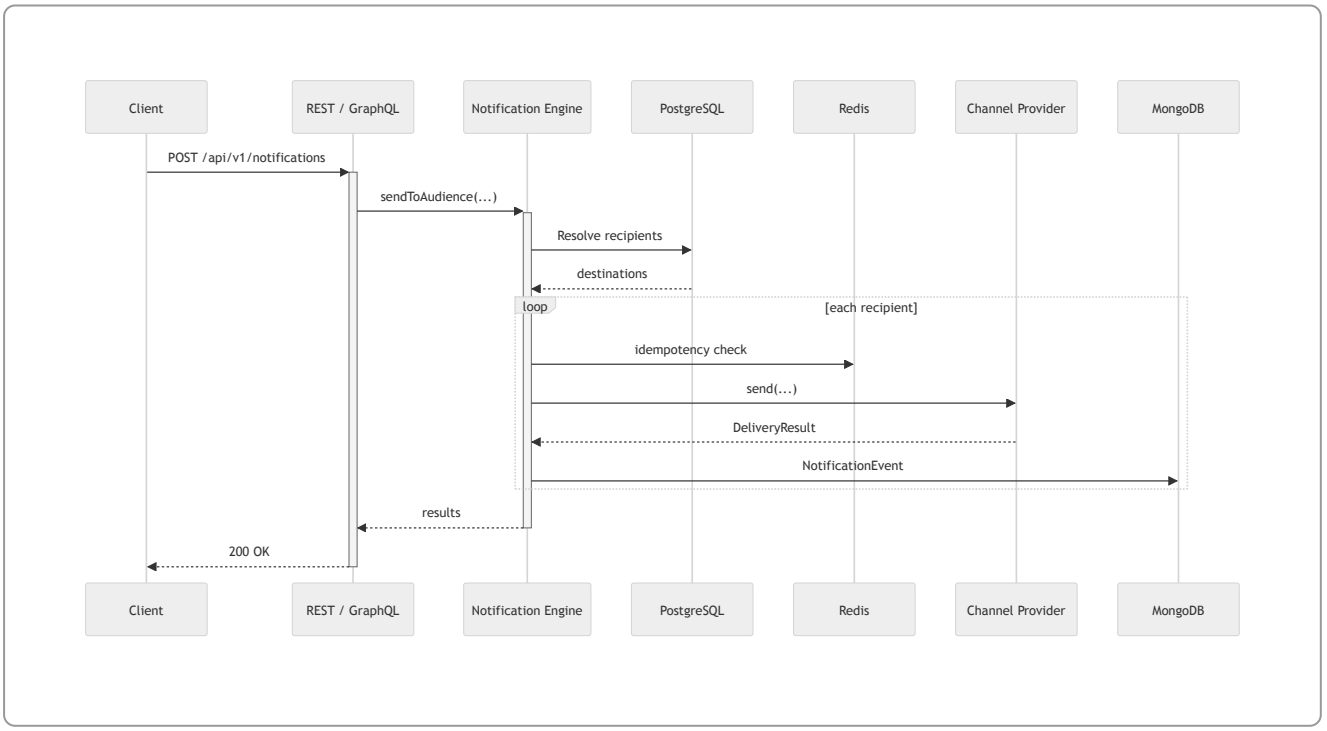
4.2 MongoDB (events)

`notification_events` : request_id, channel, recipient, payload, status, sent_at, provider_response (append-only audit log).

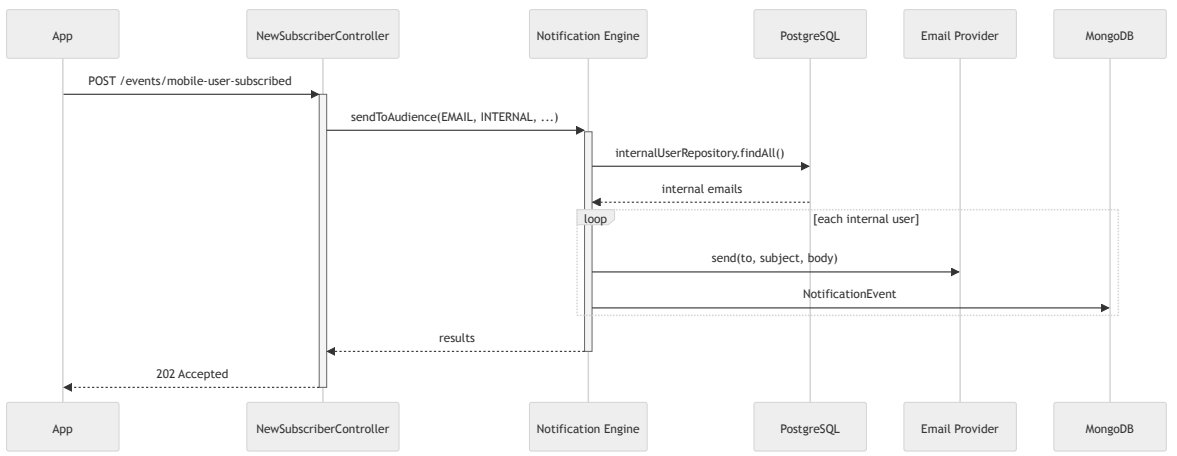
4.3 Redis

Keys for idempotency (per request/key with TTL); optional rate-limit counters.

5. Interaction: Send immediate notification

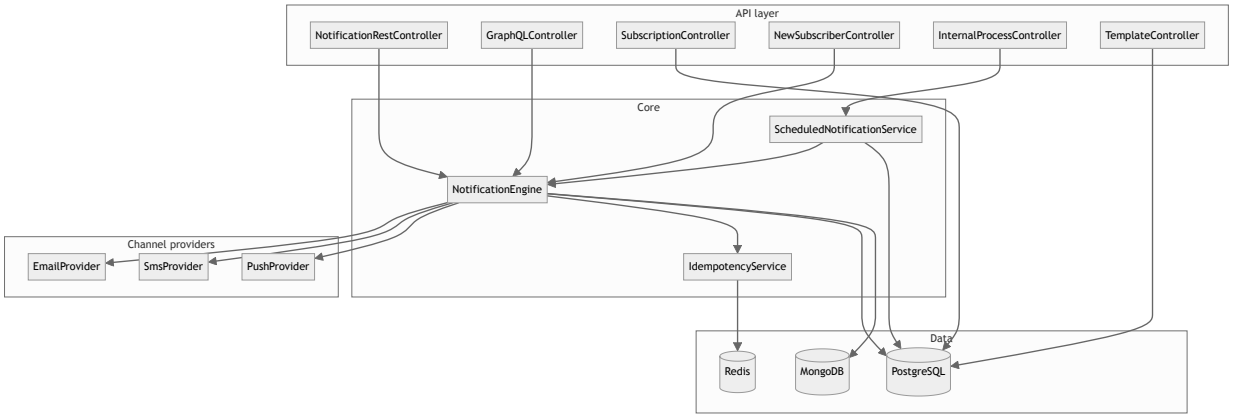


6. Interaction: New mobile subscriber → email to internal users



7. Interaction: Scheduled notifications (Scheduler Service)

8. Notification Service — layers and data flow



9. API summary

Method	Path	Description
POST	/api/v1/notifications	Send immediate notification
POST	/api/v1/notifications/scheduled	Create scheduled notification
GET	/api/v1/notifications/scheduled	List scheduled (optional ?status=)
DELETE	/api/v1/notifications/scheduled/{id}	Cancel scheduled
POST	/api/v1/events/mobile-user-subscribed	New mobile subscriber → email to internal
POST	/api/v1/subscriptions	Register subscription
GET	/api/v1/subscriptions	List subscriptions
POST	/api/v1/internal/process-due-scheduled	Internal: process due (Scheduler Service)
POST/GET	/api/v1/templates	Create / list templates

GraphQL: mutations `sendNotification`, `createScheduledNotification`, `cancelScheduledNotification`; queries `scheduledNotifications`, `subscriptions`.

10. Deployment & scalability

- **Notification Service:** Stateless; run ≥2 instances behind a load balancer (e.g. AWS ALB, Azure App Gateway). TLS 1.2+.
- **Scheduler Service:** Run one instance (or use a distributed lock if multiple).
- **PostgreSQL:** Primary + read replicas; connection pooling (HikariCP).
- **MongoDB:** Replica set for event log.
- **Redis:** Cluster or managed (ElastiCache / Azure Cache).
- **Secrets:** AWS Secrets Manager / Azure Key Vault; no secrets in code.

End of document. To save as PDF: File → Print → Save as PDF.